NUnit Hands-on – Season Converter (FourSeasonsLib)

Unit testing with NUnit — TestCaseSource, Single Assertion Rule, Assert.That()

1. Objective

Create a unit test project using the NUnit framework for the FourSeasonsLib business logic, which accepts a month name and returns the corresponding season. The goal is to write the minimum number of test methods (using **TestCaseSource**) instead of one test method per execution path, following the Single Assertion Rule and using **Assert.That()**.

Season	Month	Climate
Spring	February – March	Sunny and pleasant
Summer	April – June	Hot
Monsoon	July – September	Wet, hot and humid
Autumn	September – November	Pleasant
Winter	December – January	Very cool

Note: September appears in both Monsoon and Autumn in the given matrix. I assigned September to Monsoon in the implementation below (it is listed there first) — every month is covered exactly once.

2. Solution Structure

Added a Class Library project **ConverterLib.Tests** to the existing solution, alongside the source project **ConverterLib** (renamed test class file to SeasonConverterTests.cs), and added references to ConverterLib, NUnit, NUnit3TestAdapter and Moq via NuGet.

Solution Explorer – FourSeasonsLib

```
Solution 'FourSeasonsLib' (2 of 2 projects)
  ConverterLib
    SeasonConverter.cs
  ConverterLib.Tests
    Dependencies
      • NUnit
      • NUnit3TestAdapter
      • Moq
      • ConverterLib (project reference)
    SeasonConverterTests.cs
    SeasonTestData.cs
```

3. Source Code – ConverterLib

SeasonConverter.cs

```
using System;

namespace ConverterLib
{
    public enum Season
    {
        Spring,
        Summer,
        Monsoon,
        Autumn,
        Winter
    }

    public class SeasonConverter
    {
        public Season GetSeason(string month)
        {
            if (string.IsNullOrWhiteSpace(month))
                throw new ArgumentException("Month name cannot be null or empty", nameof(month));

            switch (month.Trim().ToLower())
            {
                case "february":
                case "march":
                    return Season.Spring;
                case "april":
                case "may":
                case "june":
                    return Season.Summer;
                case "july":
                case "august":
                case "september":
                    return Season.Monsoon;
                case "october":
                case "november":
```

```
                    return Season.Autumn;
                case "december":
                case "january":
                    return Season.Winter;
                default:
                    throw new ArgumentException($"'{month}' is not a valid month name", nameof(month));
            }
        }
    }
}
```

4. Test Code – Straightforward TestCaseSource

Test case data is supplied by a static property in the **same** test class.

SeasonConverterTests.cs

```
using System.Collections;
using ConverterLib;
using NUnit.Framework;

namespace ConverterLib.Tests
{
    [TestFixture]
    public class SeasonConverterTests
    {
        private SeasonConverter _sut;

        [SetUp]
        public void SetUp()
        {
            _sut = new SeasonConverter();
        }

        // Straightforward way: TestCaseSource pointing to a static
        // property inside this same test class
        private static IEnumerable SeasonTestCases
        {
            get
            {
                yield return new TestCaseData("February").Returns(Season.Spring);
                yield return new TestCaseData("March").Returns(Season.Spring);
                yield return new TestCaseData("April").Returns(Season.Summer);
                yield return new TestCaseData("May").Returns(Season.Summer);
                yield return new TestCaseData("June").Returns(Season.Summer);
                yield return new TestCaseData("July").Returns(Season.Monsoon);
                yield return new TestCaseData("August").Returns(Season.Monsoon);
                yield return new TestCaseData("September").Returns(Season.Monsoon);
                yield return new TestCaseData("October").Returns(Season.Autumn);
                yield return new TestCaseData("November").Returns(Season.Autumn);
                yield return new TestCaseData("December").Returns(Season.Winter);
                yield return new TestCaseData("January").Returns(Season.Winter);
            }
        }

        [Test]
        [TestCaseSource(nameof(SeasonTestCases))]
        public Season GetSeason_ValidMonth_ReturnsExpectedSeason(string month)
        {
            return _sut.GetSeason(month);
        }

        // Single Assertion Rule + Assert.That()
        [Test]
        public void GetSeason_InvalidMonth_ThrowsArgumentException()
        {
            Assert.That(() => _sut.GetSeason("Foo"), Throws.ArgumentException);
        }
    }
}
```

5. Test Code – Alternate TestCaseSource

Same scenario, but test data now comes from a **separate class** and returns `TestCaseData` from a method, each case named explicitly with `.SetName()`.

SeasonTestData.cs

```
using System.Collections.Generic;
using NUnit.Framework;

namespace ConverterLib.Tests
{
    public static class SeasonTestData
    {
        public static IEnumerable<TestCaseData> Cases()
        {
```

```
            var expected = new Dictionary<string, Season>
            {
                { "February", Season.Spring },  { "March",     Season.Spring  },
                { "April",    Season.Summer },  { "May",       Season.Summer  },
                { "June",     Season.Summer },  { "July",      Season.Monsoon },
                { "August",   Season.Monsoon }, { "September", Season.Monsoon },
                { "October",  Season.Autumn },  { "November",  Season.Autumn  },
                { "December", Season.Winter },  { "January",   Season.Winter  }
            };

            foreach (var kvp in expected)
                yield return new TestCaseData(kvp.Key)
                                .Returns(kvp.Value)
                                .SetName($"GetSeason_{kvp.Key}_Returns{kvp.Value}");
        }
    }
}
```

SeasonConverterTests.cs (added method)

```
        // Alternate way: TestCaseSource pointing to a method
        // on an external class
        [Test]
        [TestCaseSource(typeof(SeasonTestData), nameof(SeasonTestData.Cases))]
        public Season GetSeason_UsingExternalSource_ReturnsExpectedSeason(string month)
        {
            return _sut.GetSeason(month);
        }
```

6. Run the Tests

Built the solution and ran all tests from Test Explorer (Test > Run All Tests).

Test Explorer

Test Run: 25 total | **25 passed** | 0 failed | 0 skipped

✔ SeasonConverterTests
  ✔ GetSeason_ValidMonth_ReturnsExpectedSeason (12)
  ✔ GetSeason_UsingExternalSource_ReturnsExpectedSeason (12)
  ✔ GetSeason_InvalidMonth_ThrowsArgumentException

All 25 test cases pass with only two [Test] methods in code (plus the exception test) — this satisfies the "minimum code, not one test per path" requirement.

7. Break the Test

Modified `SeasonConverter.cs` to introduce a bug: changed `case "february":` to incorrectly `return Season.Summer;` instead of `Season.Spring;`, then reran the tests.

Test Explorer

Test Run: 25 total | **23 passed** | **2 failed** | 0 skipped

✘ GetSeason_ValidMonth_ReturnsExpectedSeason("February")
  Expected: Spring But was: Summer
✘ GetSeason_UsingExternalSource_ReturnsExpectedSeason("February")
  Expected: Spring But was: Summer
✔ (remaining 23 test cases)

8. Fix and Rerun

Reverted the change in `SeasonConverter.cs` back to `return Season.Spring;` for February, rebuilt, and ran the tests again.

Test Explorer

Test Run: 25 total | **25 passed** | 0 failed | 0 skipped

9. Observation

Using **TestCaseSource** (both the in-class property and the external-class method) made it possible to cover all 12 month → season execution paths with a single parameterised [Test] method each, instead of 12 separate test methods, keeping the test code minimal while still exercising every path. Breaking the source logic immediately caused the matching test cases to fail with a clear expected/actual message, and reverting the fix restored a fully green run — confirming the tests actually validate the business rule rather than passing trivially.